Homework 4

February 26, 2026

ORF307 Homework 4

Due: Friday, February 27th, 2026 9:00 pm ET

- The jupyter file is available on Canvas,
- Please export your code with output as `pdf`.
- If there is any additional answers, please combine them as **ONE** pdf file before submitting to the Gradescope.

Q1 Fitting a line with different norms (30 pts)

[5]:
```
import pandas as pd
import numpy as np
from scipy.optimize import linprog
# df = pd.read_csv('/content/drive/MyDrive/HW4Q1.csv')
df = pd.read_csv('HW4Q1.csv')
y = df['y'].values
t = df['t'].values
```

(a) (10 pts)*Least-squares:* select α and β by minimizing

$$\sum_{i=1}^{42}(y_i - \alpha - \beta t_i)^2.$$

[6]:
```
n = 42

X = np.column_stack(
    [np.ones(n), t]
)

theta = np.linalg.inv(X.T@X)@X.T@y

alpha_ls = theta[0]
beta_ls = theta[1]

print(alpha_ls, beta_ls)
```

```
4.184072009206105 0.5754837533561792
```

(b) (10 pts) ℓ_1-*norm approximation:* select α and β by minimizing

$$\sum_{i=1}^{42} |y_i - \alpha - \beta t_i|.$$

[7]:
```
c = np.zeros(2 + n)
c[2:] = 1.0  # minimize sum(u_i)

A = []
b = []

for i in range(n):
    # constraint: y_i - alpha - beta t_i <= u_i
    # => y_i - alpha - beta t_i - u_i <= 0
    # => -alpha - beta t_i - u_i <= -y_i
    row1 = np.zeros(2 + n)
    row1[0] = -1.0
    row1[1] = -t[i]
    row1[2 + i] = -1.0
    A.append(row1)
    b.append(-y[i])

    # constraint: -(y_i - alpha - beta t_i) <= u_i
    # => -y_i + alpha + beta t_i - u_i <= 0
    # => alpha + beta t_i - u_i <= y_i
    row2 = np.zeros(2 + n)
    row2[0] = 1.0
    row2[1] = t[i]
    row2[2 + i] = -1.0
    A.append(row2)
    b.append(y[i])

A = np.array(A)
b = np.array(b)

bounds = [(None, None), (None, None)] + [(0.0, None)] * n  # u_i >= 0

res_l1 = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method="highs")
alpha_l1, beta_l1 = res_l1.x[0], res_l1.x[1]
print("L1  :", alpha_l1, beta_l1)
```

```
L1  : 4.94589136112113 0.9716417486421411
```

(c) (10 pts) ℓ_∞-*norm approximation:* select α and β by minimizing

$$\max_{i=1,\ldots,42} |y_i - \alpha - \beta t_i|.$$

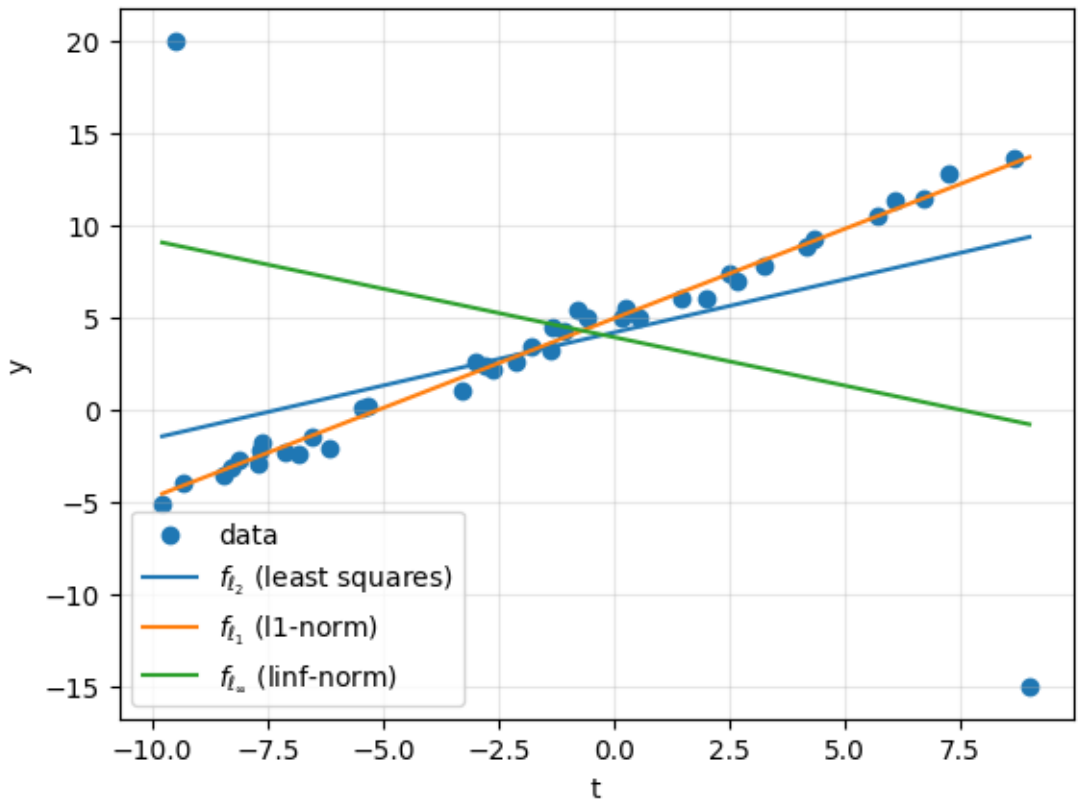
[12]:
```
c = np.array([0.0, 0.0, 1.0])  # we want to minimize c[2] = z
A = []
b = []

for i in range(n):
    row1 = np.array([-1.0, -t[i], -1.0])
    A.append(row1)
    b.append(-y[i])

    row2 = np.array([1.0, t[i], -1.0])
    A.append(row2)
    b.append(y[i])

A = np.array(A)
b = np.array(b)

bounds = [(None, None), (None, None), (0.0, None)]

res_linf = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method="highs")
alpha_linf, beta_linf, z_star = res_linf.x
print("Linf:", alpha_linf, beta_linf, ", where z* =", z_star)
```

```
Linf: 3.933516203310256 -0.5249030737747711 , where z* = 14.209388539337317
```

Find the optimal values of α and β for each of the three optimization criteria. This yields three linear functions $f_{\text{ls}}(t), f_{\ell_1}(t), f_{\ell_\infty}$. Plot the 42 data points, and the three functions f. What do you observe?

[20]:
```
import numpy as np
import matplotlib.pyplot as plt

idx = np.argsort(t)
ts = t[idx]
ys = y[idx]

f_l2   = alpha_ls   + beta_ls   * ts
f_l1   = alpha_l1   + beta_l1   * ts
f_linf = alpha_linf + beta_linf * ts

plt.figure()
plt.scatter(t, y, marker='o', label='data')

plt.plot(ts, f_l2,   label=r'$f_{\ell_2}$ (least squares)')
plt.plot(ts, f_l1,   label=r'$f_{\ell_1}$ (l1-norm)')
plt.plot(ts, f_linf, label=r'$f_{\ell_\infty}$ (linf-norm)')

plt.xlabel('t')
plt.ylabel('y')
```

```
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



it looks like L1-norm approximation fits the data best (most robust to the 2 clear outliers). least squares does a decent job, fitting the general trend of the data, but taking into some account the outliers. linf-norm does not fit the data well at all. it minimizes the worst residual, meaning the outliers have a large effect on the its function f.

Q2 Linear programs with absolute values (30 pts)

[36]:
```
from IPython.display import Image, display

display(Image("2a,b.jpg", width=600), Image("2c.jpg", width=600))
```

Q2 minimize $2x_1 + 3|x_2 - 10|$

constraint $|x_1 + 2| + |x_2| \le 5$

a)

→ let $u \ge 0$ st $u \ge 3x_2 - 10$, $u \ge -3x_2 + 10$

let $v \ge 0$ st $v \ge x_1 + 2$, $v \ge -x_1 - 2$

let $w \ge 0$ st $w \ge x_2$, $w \ge -x_2$

→ the problem then becomes : minimize $2x_1 + 3u$

subject to : $v + w \le 5$ $\quad u \ge 3x_2 - 10$, $\quad v \ge x_1 + 2$ $\quad w \ge x_2$

$u, v, w \ge 0$ $\quad u \ge -3x_2 + 10$ $\quad v \ge -x_1 - 2$ $\quad w \ge -x_2$

b) standard form : $\min(c^T x)$ st $Ax \le b$

need to add slack variables to make the inequality constraints into equality constraints → break up $|\ |$ into piecewise $(x_1 = x_1^+ - x_1^-,\ x_1^+ \ge 0,\ x_1^- \ge 0)$

→ the problem becomes : minimize $2x_1^+ - 2x_1^- + 3u$

subject to : $u + x_2^+ - x_2^- - s_2 = 10$, $\quad -u + x_2^+ - x_2^- + s_1 = 10$

$v - x_1^+ + x_1^- - s_3 = 10$, $\quad -v - x_1^+ + x_1^- + s_4 = 2$

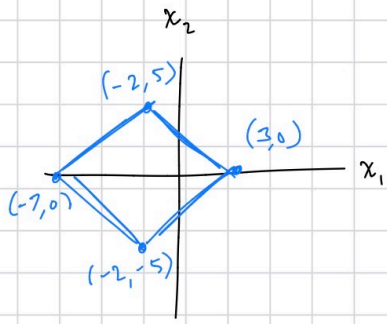
$-w + x_2^+ - x_2^- + s_5 = 0$, $\quad -w - x_2^+ + x_2^- + s_6 = 0$

$v + w + s_7 = 5$

$x^+ \ge 0$, $x^- \ge 0$, $\{u, v, w\} \ge 0$, $s_i \ge 0$

where s_i are slack variables to "construct" equalities

c)



$\|x_1 + 2\| + \|x_2\| \le 5$

means $\|x_2\| \le 5 \rightarrow x_2 \in \{-5, 5\}$

$\rightarrow x_2 - 10 \le 5 - 10 = -5 < 0$

$\rightarrow \|x_2 - 10\| = 10 - x_2$

so minimizing $2x_1 + 3\|x_2 - 10\|$ is acc equal to $2x_1 + 3(10 - x_2)$

$= 2x_1 + 30 - 3x_2$,

which is the same as minimizing

$2x_1 - 3x_2$

we only need to check crit points (corners), so

(3, 0): 6

(-2, 5): $-6 - 15 = -21$ ← min, so minimum @ $(x_1, x_2) = (-2, 5)$

(-2, -5): $-6 + 15 = 9$

(-7, 0): -14

where $2x_1 + 3\|x_2 - 10\|$

$= -4 + 3(5) = 11$

Q3 Optimal Illumination (40 pts)

The patches are line segments; the ith patch is given by $[v_i, v_{i+1}]$ where $v_1, \dots, v_{n+1} \in \mathbf{R}^2$. The variables in the problem are the lamp powers $p_1, \dots, p_m$, which can vary between 0 and 1. The illumination at (the midpoint of) patch i is denoted I_i. We will use a simple model for the illumination:

$$I_i = \sum_{j=1}^{m} a_{ij} p_j, \qquad a_{ij} = r_{ij}^{-2} \max\{\cos\theta_{ij}, 0\},$$

where r_{ij} denotes the distance between lamp j and the midpoint of patch i, and θ_{ij} denotes the angle between the upward normal of patch i and the vector from the midpoint of patch i to lamp j,

as shown in the figure. This model takes into account "self-shading" (*i.e.*, the fact that a patch is illuminated only by lamps in the halfspace it faces) but not shading of one patch caused by another. Of course we could use a more complex illumination model, including shading and even reflections. This just changes the matrix relating the lamp powers to the patch illumination levels.

The problem is to determine lamp powers that make the illumination levels close to a given desired illumination level I_{des}, subject to the power limits $0 \leq p_i \leq 1$.

(a) (10 pts) Suppose we use the maximum deviation

$$\phi(p) = \max_{k=1,\ldots,n} |I_k - I_{\text{des}}|$$

as a measure for the deviation from the desired illumination level. Formulate the illumination problem using this criterion as a linear optimization problem.

[37]:
```
Image("3a.png", width=600)
```

[37]:

Q3

p_j betw. $\{0, 1\}$

a) Illumination $I_i = \sum_{j=1}^{m} a_{ij} p_j$ $\quad a_{ij} = r_{ij}^{-2} \max\{\cos\theta_{ij}, 0\}$

$\phi(p) = \max_{k=1,\ldots,n} \| I_k - I_{des} \|$ = deviation from desired illumination level

Introduce decision variable $s \in \mathbb{R}$

→ we want to minimize : s

subject to : $s \geq \sum_{j=1}^{m} a_{kj} p_j - I_{des} \geq -s \quad \forall k = 1, \ldots, n$

$1 \geq p_j \geq 0 \quad \forall j = 1, \ldots, n$

(b) (20 pts) There are several suboptimal approaches based on weighted least-squares. You are given A (where the entries of A are a_{ij} in the model for illumination) and I_{des}. Import the file `HW4Q3.csv` using the following lines of code, you might have to change `HW4Q3.csv` to the corresponding filepath on your machine/Google Drive

[38]:
```
A_pd = pd.read_csv('HW4Q3.csv', header=None)
A = A_pd.to_numpy()
I_des = 2
```

[46]:
```
import cvxpy as cp
```

```python
def evalPhi(p, A, I_des):
    I_k = A@p
    return max(abs(I_k - I_des))
# b1
m = A.shape[1]
n = A.shape[0]
p = cp.Variable((m,1))
obj_b1 = cp.Minimize(cp.sum_squares(A@p - I_des))
prob_b1 = cp.Problem(obj_b1)
prob_b1.solve()
print('There are some entries in p greater than 1 and some less than 0: \n', p.
 ↪value )
p.value[p.value>=1] = 1
p.value[p.value<=0] = 0
print('\n Value of p after processing: \n', p.value)
print('Phi(p): ', evalPhi(p.value, A, I_des))
print('\n')
# b2
mu=7.3
obj_b2 = cp.Minimize(cp.sum_squares(A@p-I_des) + mu*cp.sum_squares(p-0.5))
prob_b2 = cp.Problem(obj_b2)
prob_b2.solve()
print('Value of p: \n', p.value)
print('Phi(p): ', evalPhi(p.value, A, I_des))
```

```
There are some entries in p greater than 1 and some less than 0:
 [[ 5.26966575]
 [-2.45573954]
 [ 0.52322558]
 [-8.12880821]
 [11.67060976]
 [-5.66888204]
 [-1.42315408]]

 Value of p after processing:
 [[1.        ]
 [0.        ]
 [0.52322558]
 [0.        ]
 [1.        ]
 [0.        ]
 [0.        ]]
Phi(p):  [3.24695557]


Value of p:
 [[ 0.60361662]
 [ 0.5003586 ]
```

```
 [-0.02211918]
 [ 0.34445844]
 [ 0.38777075]
 [ 0.46907505]
 [ 0.45914574]]
Phi(p):  [1.30430795]
```

[48]:
```
I_des
```

[48]: 2

[49]:
```
A
```

[49]:
```
array([[0.69483074, 0.41069986, 0.38410511, 0.15443517, 0.00916266,
        0.04803079, 0.        ],
       [0.97217829, 0.57819842, 0.75868342, 0.23550446, 0.01339488,
        0.06454695, 0.        ],
       [1.50500273, 1.22257337, 4.83611427, 1.403457  , 0.94613895,
        0.57121288, 0.63840044],
       [0.60716988, 0.72925165, 3.79641008, 1.3863946 , 1.21548896,
        0.65718704, 0.87482388],
       [1.08191029, 0.8231571 , 3.39358194, 0.78363859, 0.35580535,
        0.22030357, 0.16772274],
       [1.30344901, 1.08100541, 6.41560884, 1.27708103, 0.58669588,
        0.31324246, 0.25636096],
       [0.19613316, 0.49166131, 1.48612843, 1.61614496, 1.85952307,
        0.87030902, 1.33958934],
       [0.        , 0.05398322, 0.        , 0.49332918, 0.82940328,
        0.46359117, 0.74444901],
       [0.57121288, 0.64092806, 1.70254085, 1.16936954, 1.23286934,
        0.68288952, 0.95464769],
       [0.64926338, 0.61280701, 1.98388105, 0.95552107, 0.75256232,
        0.31659509, 0.39751472],
       [0.        , 0.10076251, 0.        , 0.42416225, 0.86887711,
        0.57294845, 0.92551863]])
```

(c) (10 pts) Using the same data as in part (b), solve the LP you derived in part (a). Compare the solution with the solutions you obtained using the (weighted) least-squares methods of part (b).

[50]:
```
p,s = cp.Variable((m,1)), cp.Variable(1)
constraints = []
for k in range(n):
    constraints += [s >= A[k,:]@p - I_des, A[k,:]@p - I_des >= -s]
constraints += [1 >= p, p >= 0]
obj = cp.Minimize(s)
prob = cp.Problem(obj, constraints)
prob.solve()
```

```python
print('Value of p: \n', p.value)
print('Phi(p): ', prob.value)
```

```
Value of p:
 [[9.99999990e-01]
 [5.70808574e-01]
 [2.65742121e-10]
 [7.83801665e-09]
 [1.95973426e-08]
 [3.70580701e-01]
 [9.99999945e-01]]
Phi(p):  1.0529389803867861
```

the result from the LP best, followed by the result computed from the least squares method. the saturated least-squares method results in the worst solution.